



IE3090

Network Programming

3rd Year, Semester I

RemoteOps - Assignment Part 1

A Remote System Monitoring and Management Tool over TCP/IP

Submitted to
Sri Lanka Institute of Information Technology

In partial fulfillment of the requirements for the
Bachelor of Science Special Honors Degree in Information Technology

7th October 2026

Declaration

I certify that this report does not incorporate without acknowledgement, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief it does not contain any material previously published or written by another person, except where due reference is made in text.

Registration Number : IT24103381

Name : Jayarathne A W M R

Table of Contents

1. Registration Number and Personalised Values	1
2. Architecture Overview	2
2.1 Components	2
2.2 Diagram	2
2.3 Concurrency model: one thread per connection.....	3
2.4 Shared state and thread safety	3
3. Protocol Implementation	4
3.1 Framing	4
3.2 Protocol evidence (full request/response exchange)	5
4. Personalisation Proof	6
4.1 Agent listening on port 9410.....	6
4.2 Personalised log file.....	6
4.3 Personalised storage path.....	6
5. Annotated Code	7
5.1 Personalised constants and session structure	7
5.2 Connection handling and concurrency	8
5.3 Line framing and byte counting	9
5.4 Authentication and command dispatch	10
5.5 SYSINFO and LISTPROC	11
5.6 Restricted EXEC	12
5.7 File upload and download	13
5.8 UDP monitoring loop.....	14
5.9 Logging.....	15
5.10 Controller	16
6. Execution and Output Screenshots	18
6.1 Agent starting up.....	18
6.2 Successful AUTH	18
6.3 SYSINFO, LISTPROC and EXEC	18
6.4 Successful PUT and GET	19
6.5 UDP monitoring stream	19
6.6 Error cases handled gracefully	20
6.7 Five simultaneous Controllers	20
7. Testing Summary	22

8. Design Rationale and Assumptions.....	24
8.1 Key design decisions.....	24
8.2 Assumptions where the brief leaves a choice	24
8.3 Use of AI assistance.....	25
9. Optional Extension and Limitations	26
10. Build and Run Instructions	27

List of Figures

Figure 2.1: RemoteOps architecture (Controller, Agent, threads and channels)	2
Figure 3.1: Full protocol session showing the SID tag	5
Figure 4.1: Agent listening on port 9410	6
Figure 4.2: Contents of remoteops_IT24103381.log	6
Figure 4.3: Uploaded file in ./agentfiles/IT24103381	6
Figure 5.1: Constants and session_t structure (agent_381.c)	7
Figure 5.2: main() accept loop and client_thread()	8
Figure 5.3: read_line() and read_some()	9
Figure 5.4: handle_line() authentication and dispatch	10
Figure 5.5: get_sysinfo() and get_procs()	11
Figure 5.6: WHITELIST table and run_fixed()	12
Figure 5.7: valid_filename(), do_put() and do_get()	13
Figure 5.8: monitor_thread() and stop_monitor()	14
Figure 5.9: log_msg()	15
Figure 5.10: UDP receiver and do_put() (controller_381.c)	16
Figure 6.1: Agent starting up	18
Figure 6.2: Successful AUTH	18
Figure 6.3: SYSINFO, LISTPROC and EXEC	19
Figure 6.4: Successful PUT and GET	19
Figure 6.5: UDP monitoring stream	20
Figure 6.6: Error cases handled gracefully	20
Figure 6.7: Five simultaneous Controllers	21

1. Registration Number and Personalised Values

My registration number is **IT24103381**. It has the same format as the example in the assignment brief (the letters IT followed by eight digits), so every formula in section 2.4 of the brief applies directly to the numeric part **24103381**. The table below shows each value, the calculation, and where it appears in my implementation.

Item	Calculation from IT24103381	Value
Registration number	Given	IT24103381
Agent listening port	$7000 + \text{first four digits of } 24103381 (2410) = 7000 + 2410$	9410
Source file names	Last three digits of 24103381 (381)	agent_381.c, controller_381.c, Makefile_381
Session ID (SID) tag	Last four digits (3381), reversed	SID:1833
Authentication token	“OPS-” followed by the last four digits	OPS-3381
Log file name	remoteops_<full registration number>.log	remoteops_IT24103381.log
File storage path	./agentfiles/<full registration number>/<filename>	./agentfiles/IT24103381/<filename>
Submission archive	IE3090_<full registration number>.zip	IE3090_IT24103381.zip

In the code these values are defined once as constants at the top of agent_381.c (LISTEN_PORT, SID, AUTH_TOKEN, LOG_FILE, STORE_DIR) and are used everywhere else, so there is no hard-coded copy of any value elsewhere in the program (see Section 5.1).

2. Architecture Overview

2.1 Components

RemoteOps has two programs that talk over a TCP control channel and a secondary UDP monitoring channel.

- **Agent** (`agent_381.c`) is the server that runs on the “managed” machine. It listens on TCP port 9410, authenticates each Controller, executes the commands of the protocol (system information, process list, whitelisted commands, file upload and download) and can stream periodic system statistics over UDP. It writes a timestamped log of every connection, command and file transfer to `remoteops_IT24103381.log` and stores uploaded files under `./agentfiles/IT24103381/`.
- **Controller** (`controller_381.c`) is the interactive client used by the administrator. It reads commands typed by the user, sends them to the Agent over TCP, and prints each response. It performs the file transfers for PUT and GET (adding the file size and the raw bytes), reports the transfer throughput in bytes per second, and runs a small UDP receiver thread that prints the monitoring datagrams after `MONITOR START <udp_port>`.

2.2 Diagram

Figure 2.1 shows the two programs, the threads inside them, and the two channels between them. The red arrow is the TCP control channel on port 9410; every reply on it ends with the tag `SID:1833`. The green path is the UDP monitoring stream, sent by a monitor thread inside the Agent to the UDP port that the Controller announced in `MONITOR START`.

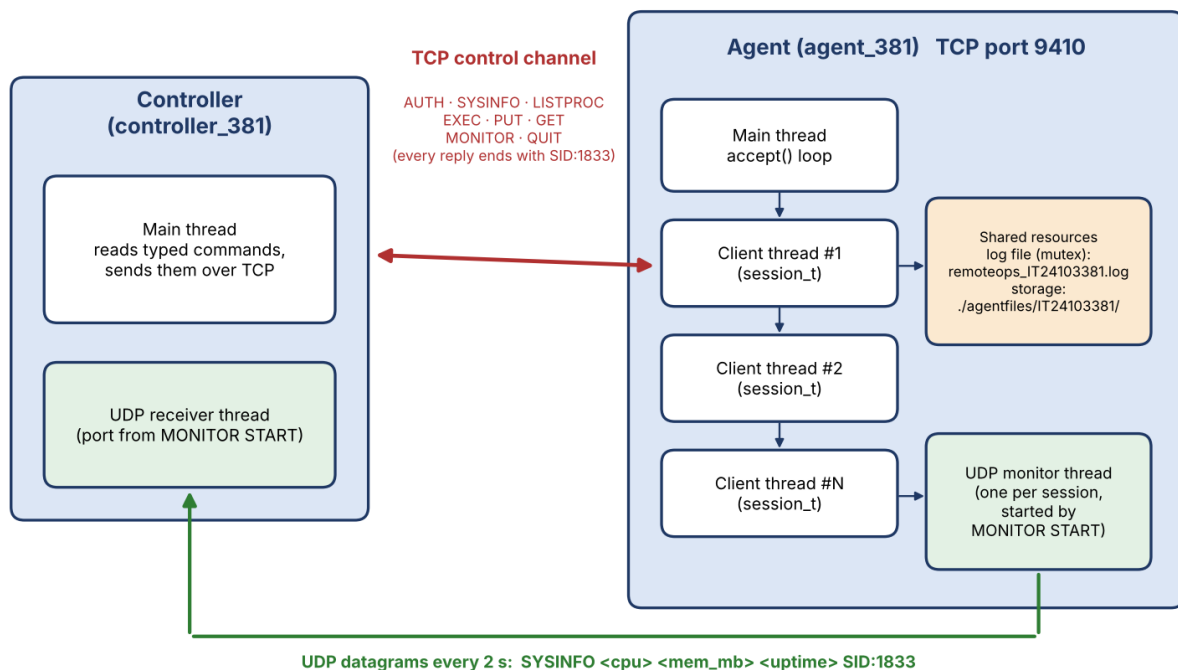


Figure 2.1: RemoteOps architecture (Controller, Agent, threads and channels)

2.3 Concurrency model: one thread per connection

The Agent uses **one POSIX thread per Controller connection**. The main thread only runs the `accept()` loop. For every accepted connection it allocates a private `session_t`, creates a detached thread running `client_thread()`, and immediately goes back to `accept()`. The thread reads and executes commands for that one Controller until it sends QUIT or disconnects, then stops its monitor thread, closes the socket, logs the disconnect and frees its session.

I chose threads over the two alternatives allowed by the brief for these reasons:

- **Simple, blocking code.** Each thread can block in `recv()` while waiting for its own Controller without affecting anybody else. The framing code (Section 3.1) can therefore be written as a straight loop instead of an explicit state machine per connection, which is what `select()/poll()` would require for partial lines and large PUT/GET transfers.
- **Cheaper and easier than `fork()`.** Threads share the log file and the address space, so no inter-process communication is needed for logging, and creating a thread is lighter than creating a process.
- **Private session state.** Everything that belongs to one connection (socket, authentication flag, receive buffer, monitoring thread) lives in its own `session_t`, so the threads need almost no locking (Section 2.4).

The trade-off is scalability: one thread per client does not suit thousands of simultaneous Controllers. The brief requires at least five simultaneous connections on one machine, which this design meets easily (see test 20 in Section 7 and Figure 6.7).

2.4 Shared state and thread safety

- **Log file.** The only data shared by all client threads is the log file. `log_msg()` takes a mutex (`g_log_mu`) around each write so lines from different threads never interleave.
- **Per-session data.** The socket, authentication flag, receive buffer and monitor-thread handle are inside the thread's own `session_t`. The session is created with `calloc()` in the accept loop and freed by the thread when it ends.
- **Monitor stop flag.** Each session's monitor thread is stopped through an atomic integer (`mon_stop`) and then joined with `pthread_join()`, so no thread outlives its session.
- **Shutdown flag.** `g_running` is a volatile `sig_atomic_t` set by the SIGINT/SIGTERM handler. SIGPIPE is ignored and sends use MSG_NOSIGNAL, so a Controller that disappears cannot kill the Agent.
- **Uploaded files.** A PUT writes to a temporary file named with the thread id and renames it to the final name only when all bytes have arrived, so a half-uploaded file never appears under its real name and two uploads never write the same temporary file.

3. Protocol Implementation

The protocol is exactly the one given in section 2.3 of the brief: every command and response is one text line ending in `\n`, except PUT and GET, whose lines are followed by exactly `<filesize>` raw bytes. Every response the Agent sends, OK or ERR, ends with a space and `SID:1833`. The table lists each command, the function that implements it and the responses it produces.

Command	Implemented by	Success response	Error responses
AUTH <token>	handle_line()	OK AUTHENTICATED SID:1833	ERR 001 AUTH_FAILED
any command before AUTH	handle_line()	—	ERR 008 NOT_AUTHENTICATED
SYSINFO	get_sysinfo()	OK SYSINFO <cpu_load> <mem_used_mb> <uptime_sec>	—
LISTPROC	get_procs()	OK PROCS <pid:name,pid:name,...>	—
EXEC <name>	whitelist_lookup(), run_fixed()	OK EXEC_RESULT <output>	ERR 002 COMMAND_NOT_ALLOWED
PUT <filename> <filesize> + bytes	do_put()	OK FILE_RECEIVED <filename>	ERR 004 FILE_TOO_LARGE, ERR 003 BAD_FILENAME, ERR 006 BAD_REQUEST, ERR 011 STORAGE_ERROR
GET <filename>	do_get()	OK FILE_SEND <filename> <filesize> + bytes	ERR 005 FILE_NOT_FOUND
MONITOR START <udp_port>	handle_line(), monitor_thread()	OK MONITOR_STARTED	ERR 007 MONITOR_ALREADY_RUNNING, ERR 006 BAD_REQUEST
MONITOR STOP	stop_monitor()	OK MONITOR_STOPPED	—
QUIT	handle_line()	OK BYE	—
anything else	handle_line()	—	ERR 009 UNKNOWN_COMMAND, ERR 010 LINE_TOO_LONG, ERR 012 INTERNAL_ERROR

The EXEC whitelist is fixed to the five names in the brief: DATE, UPTIME, DISKFREE, HOSTNAME and WHOAMI. Anything else, including attempts such as `EXEC rm` or `EXEC DATE;id`, is rejected with ERR 002. Because the output of a command can span several lines but a response must be one line, line breaks in command output are replaced by `|`.

3.1 Framing

TCP is a byte stream, not a message stream, so one `recv()` may return half a line, several lines, or a line followed by the first bytes of a file. The Agent handles this with a per-session receive buffer (`rbuf`, with a read position `rpos` and a fill level `rlen`):

- `read_line()` scans the buffer for `\n`, strips an optional `\r`, and calls `recv()` again only when the buffer is empty. A partial line is therefore completed by later `recv()` calls, and several lines in one buffer are returned one at a time because the leftover bytes stay in `rbuf`.
- For PUT, `read_some()` first returns the bytes that are already in the buffer (the start of the file may have arrived together with the header line) and only then calls `recv()`, so exactly `<filesize>` bytes are consumed regardless of how many `recv()` calls it takes.
- For GET, `send_all()` loops until all bytes are written, so partial `send()` calls cannot truncate a file.
- If a PUT is rejected (too large or bad file name), the Agent first **drains** the announced number of bytes and only then sends the error, so the file's bytes are never misread as commands.
- A line longer than 1024 bytes is answered with `ERR 010 LINE_TOO_LONG` and the connection is closed.

3.2 Protocol evidence (full request/response exchange)

Figure 3.1 is a captured session on my own machine. It shows EXEC DATE, EXEC HOSTNAME, a PUT and a GET of `testfile.txt`, and QUIT, with the `SID:1833` tag at the end of every response and the throughput line printed by the Controller. (The long LISTPROC output at the top is cut off by the edge of the screenshot.)

```
manu@localhost:~/IE3090_RemoteOps_IT24103381$ echo "hello remoteops" > testfile.txt
manu@localhost:~/IE3090_RemoteOps_IT24103381$ ./controller_381
Connected to 127.0.0.1:9410. Start with: AUTH <token>
remoteops> AUTH OPS-3381
OK AUTHENTICATED SID:1833
remoteops> SYSINFO
OK SYSINFO 0.01 1206 2594 SID:1833
remoteops> LISTPROC
OK PROCs 1:systemd,2:kthreadd,3:pool_workqueue_release,4:kworker/R-rcu_gp,5:kworker/R-sync_wq,6:kworker/R-kvfree_r
cu_reclaim,7:kworker/R-slub_flushwq,8:kworker/R-netns,12:kworker/u512:0-events_unbound,13:kworker/R-mm_percpu_wq,1
4:kworker/u512:1-ipv6_addrconf,15:ksoftirqd/0,16:rcu_preempt,17:rcu_exp_par_gp_kthread_worker/1,18:rcu_exp_gp_kthr
ead_worker,19:migration/0,20:idle_inject/0,21:cpuhp/0,22:cpuhp/1,23:idle_inject/1,24:migration/1,25:ksoftirqd/1,27
:kworker/1:0H-events_highpri,28:cpuhp/2,29:idle_inject/2,30:migration/2,31:ksoftirqd/2,32:kworker/2:0-events_freez
able_pwr_efficient,33:kworker/2:0H-events_highpri,34:cpuhp/3,35:idle_inject/3,36:migration/3,37:ksoftirqd/3,41:kwo
rker/u514:0-events_unbound,46:kworker/u513:1-events_unbound,47:kdevtmpfs,48:kworker/R-inet_frag_wq,49:rcu_tasks_kt
hread,50:rcu_tasks_rude_kthread,51:rcu_tasks_trace_kthread,52:kauditd,53:khungtaskd,54:oom_reaper,55:kworker/R-wri
teback,56:kcompactd0,57:ksmd,58:khugepaged,59:kworker/R-cryptd,60:kworker/R-kblockd,61:kworker/R-blkcq_punt_bio,62
:kworker/R-kintegrityd,63:irq/9-acpi,66:kworker/R-tpm_dev_wq,67:kworker/R-md_bitmap,68:kworker/R-md,69:kworker/R-e
dac-poller,70:watchdogd,71:kworker/2:1H-kblockd,73:kworker/u513:2-events_unbound,74:kswapd0,91:kworker/R-kthrotld,
96:irq/24-pciehp,97:irq/25-pciehp,98:irq/26-pciehp,99:irq/27-pciehp,100:irq/28-pciehp,101:irq/29-pciehp,102:irq/30
-pciehp,103:irq/31-pciehp,104:irq/32-pciehp,105:irq/33-pciehp,106:irq/34-pciehp,107:irq/35-pciehp,108:irq/36-pcieh
p,109:irq/37-pciehp,110:irq/38-pciehp,111:irq/39-pciehp,112:irq/40-pciehp,113:irq/41-pciehp,114:irq/42-pciehp,115:
irq/43-pciehp,116:irq/44-pciehp,117:irq/45-pciehp,118:irq/46-pciehp,119:irq/47-pciehp,120:irq/48-pciehp,121:irq/49
-pciehp,122:irq/50-pciehp,123:irq/51-pciehp,124:irq/52-pciehp,125:irq/53-pciehp,126:irq/54-pciehp,127:irq/55-pcieh
p,128:kworker/R-acpi_thermal_pm,129:kworker/R-kmpath_rdacd,130:kworker/R-kaluald,132:kworker/R-mld,134:kworker/R-ip
v6_addrconf,139:kworker/R-kstrp,146:kworker/u515:0,147:kworker/u516:0-ttm,148:kworker/u517:0,188:kworker/u514:4-fl
ush-253:0,204:kworker/u514:7-events_unbound,212:kworker/u513:3-events_unbound,290:kworker/0:1H-xfs-log/dm-0,302:kw
orker/3:1H,342:kworker/R-kmpathd,343:kworker/R-kmpath_handlerd,481:kworker/R-iscsi_conn_cleanup,485:kworker/R-tls-
strp,487:kworker/R-cnric_wq,488:bnx2i_thread/0,489:bnx2i_thread/1,490:bnx2i_thread/2,491:bnx2i_thread/3,655:kworker
/R-ata_sff,656:kworker/R-nvme-wq,657:kworker/R-nvme-reset-wq,658:kworker/R-nvme-delete-wq,659:kworker/R-nvme-auth-
wq,660:scsi_ah_0,661:kworker/R-scsi_tmf_0,662:kworker/R-mpt_poll_0,663:kworker/R-mpt_0,664:kworker/R-nvme_tcp_wq,6
65:scsi_ah_1,666:kworker/R-scsi_tmf_1,684:scsi_ah_2,685:kworker/R-scsi_tmf_2,771:kworker/R-kdflush/253:0,778:kwo
rker/R-kdflush/253:1,810:kworker/R-xfs_mnu_cache,811:kworker/R-xfs_mnu_cache,812:kworker/R-xfs-buf/dm-0,813:kworker/R-x
fs-conv/dm-0,814:kworker/R-xfs-reclaim/dm-0,815:kworker/R-xfs-blockgc/dm-0,816:kworker/R-xfs-inodegc/dm-0,817:kwor
ker/R-xfs-log/dm-0,818:kworker/R-xfs-cil/dm-0,819:xfsaild/dm-0,894:irq/56-vmm_vmc1,895:irq/57-vmm_vmc1,896:irq/58-
vmm_vmc1,912:systemd-journal,959:systemd-udev,1004:kworker/0:2H-kblockd,1010:kworker/R-xfs-buf/sda2,1011:kworker/R
-xfs-conv/sda2,1012:kworker/R-xfs-reclaim/sda2,1013:kworker/R-xfs-blockgc/sda2,1014:kworker/R-xfs-inodegc/sda2,10
15:kworker/R-xfs-log/sda2,1016:kworker/R-xfs-cil/sda2,1017:xfsaild/sda2,1029:irq/16-vmmgfx,1030:kworker/R-ttm,1042
:auditd,1044:sedispach,1092:dbus-broker-lau,1099:dbus-broker-lau,1100:avahi-daemon,1101:chronyd,1104:firewalld,1105:N
etworkManager,1107:irqbalance,1108:lsmd,1110:mclog,1111:polkitd,1114:rtkit-daemon,1116:accounts-daemon,1117:switc
hero-cont,1118:systemd-logind,1119:udisksd,1121:upowerd,1123:VGAuthService,1125:vmtoolsd,1135:systemd-userdbd,114
4:alsactl,1168:avahi-daemon,1188:cupsd,1190:sshd,1225:atd,1228:crond,1233:gdm,1241:ModemManager,1242:tuned,1338:tu
ned-ppd,1438:rsyslogd,1923:colord,2759:wpa_supplicant,2861:kworker/u516:2-ttm,2892:gdm-session-wor,2909:systemd,29
11:(sd-pam),2928:gnome-keyring-d,2938:gdm-wayland-ses,2944:dbus-broker-lau,2950:dbus-broker,2953:gnome-session-b,3
SID:1833
remoteops> EXEC DATE
OK EXEC_RESULT Tue Oct 6 05:57:47 PM EDT 2026 SID:1833
remoteops> EXEC HOSTNAME
OK EXEC_RESULT localhost.localdomain SID:1833
remoteops> PUT testfile.txt
OK FILE_RECEIVED testfile.txt SID:1833
[PUT] 16 bytes in 0.0423 s = 378 bytes/s
remoteops> GET testfile.txt
OK FILE_SEND testfile.txt 16 SID:1833
[GET] saved 16 bytes to dL_testfile.txt in 0.0020 s = 7837 bytes/s
remoteops> QUIT
OK BYE SID:1833
manu@localhost:~/IE3090_RemoteOps_IT24103381$
```

Figure 3.1: Full protocol session showing the `SID` tag

4. Personalisation Proof

4.1 Agent listening on port 9410

The command `ss -tlnp | grep 9410` shows agent_381 listening on TCP port 9410 on all interfaces (7000 + 2410 = 9410).

```
manu@localhost:~/IE3090_RemoteOps_IT24103381$ ss -tlnp | grep 9410
LISTEN 0      16          0.0.0.0:*    0.0.0.0:*    users:((("agent_381",pid=5964,fd=4))
manu@localhost:~/IE3090_RemoteOps_IT24103381$ ./controller_381
Connected to 127.0.0.1:9410. Start with: AUTH <token>
remoteops> AUTH OPS=3381
```

Figure 4.1: Agent listening on port 9410

4.2 Personalised log file

The Agent writes to `remoteops_IT24103381.log`. Each line has a timestamp and the client address; the first line records the personalised port, SID tag and storage path, and the lines that follow record the connection, the authentication (the token itself is never written to the log), every command, the stored and sent file, and the disconnect.

```
manu@localhost:~/IE3090_RemoteOps_IT24103381$ head -20 remoteops_IT24103381.log
[2026-10-06 09:16:03] RemoteOps Agent (IT24103381) listening on TCP port 9410, SID:1833, storage ./agentfiles/IT24103381
[2026-10-06 09:16:44] 127.0.0.1:44950 connected
[2026-10-06 09:18:31] 127.0.0.1:44950 CMD AUTH <token>
[2026-10-06 09:18:31] 127.0.0.1:44950 authentication OK
[2026-10-06 09:18:42] 127.0.0.1:44950 CMD SYSINFO
[2026-10-06 09:18:58] 127.0.0.1:44950 CMD LISTPROC
[2026-10-06 09:19:12] 127.0.0.1:44950 CMD EXEC DATE
[2026-10-06 09:19:26] 127.0.0.1:44950 CMD EXEC HOSTNAME
[2026-10-06 09:19:40] 127.0.0.1:44950 CMD PUT testfile.txt 16
[2026-10-06 09:19:40] 127.0.0.1:44950 PUT testfile.txt 16 bytes stored at ./agentfiles/IT24103381/testfile.txt
[2026-10-06 09:19:53] 127.0.0.1:44950 CMD GET testfile.txt
[2026-10-06 09:19:53] 127.0.0.1:44950 GET testfile.txt 16 bytes sent
[2026-10-06 09:20:09] 127.0.0.1:44950 CMD QUIT
[2026-10-06 09:20:09] 127.0.0.1:44950 disconnected (QUIT)
[2026-10-06 09:33:13] 127.0.0.1:52814 connected
[2026-10-06 09:33:27] 127.0.0.1:52814 CMD AUTH <token>
[2026-10-06 09:33:27] 127.0.0.1:52814 authentication OK
[2026-10-06 09:33:38] 127.0.0.1:52814 CMD MONITOR START 5000
[2026-10-06 09:33:38] 127.0.0.1:52814 MONITOR stream started -> udp port 5000 (every 2000 ms)
[2026-10-06 09:35:54] 127.0.0.1:40524 connected
manu@localhost:~/IE3090_RemoteOps_IT24103381$ |
```

Figure 4.2: Contents of `remoteops_IT24103381.log`

4.3 Personalised storage path

The uploaded file `testfile.txt` is stored under the personalised path `./agentfiles/IT24103381/`.

```
[2026-10-06 09:35:54] 127.0.0.1:40524 connected
manu@localhost:~/IE3090_RemoteOps_IT24103381$ ls -l agentfiles/IT24103381/
total 4
-rw-r--r-- 1 manu manu 16 Oct  6 17:58 testfile.txt
manu@localhost:~/IE3090_RemoteOps_IT24103381$ |
```

Figure 4.3: Uploaded file in `./agentfiles/IT24103381`

5. Annotated Code

The figures in this section show the key parts of `agent_381.c` and `controller_381.c`, with the original line numbers. Each is followed by a short explanation.

5.1 Personalised constants and session structure

```
/* ---- personalised constants ---- */
#define REG_NO      "IT24103381"
#define LISTEN_PORT 9410
#define SID         "1833"
#define AUTH_TOKEN  "OPS-3381"
#define LOG_FILE    "remoteops_IT24103381.log"
#define STORE_ROOT  "./agentfiles"
#define STORE_DIR   "./agentfiles/IT24103381"

/* ---- tunables ---- */
#define MAX_FILE_SIZE (10UL * 1024 * 1024) /* 10 MB upload limit */
#define DRAIN_LIMIT   (200UL * 1024 * 1024) /* never drain more */
#define MONITOR_INTERVAL_MS 2000 /* UDP stats every 2 s */
#define RBUF_SIZE      4096
#define LINE_MAX_LEN   1024
#define OUT_MAX        4096

typedef struct {
    int fd;
    struct sockaddr_in peer;
    char peer_str[64];
    int authed;
    int quit_clean;
    char rbuf[RBUF_SIZE]; /* receive buffer (framing)
    size_t rpos, rlen;
    int mon_active;
    int mon_port;
    pthread_t mon_thread;
    atomic_int mon_stop;
} session_t;
```

Figure 5.1: Constants and `session_t` structure (`agent_381.c`)

All personalised values are defined once at the top of the file: `LISTEN_PORT 9410`, `SID "1833"`, `AUTH_TOKEN "OPS-3381"`, `LOG_FILE` and `STORE_DIR`. The tunables set the 10 MB upload limit, the 200 MB limit up to which a rejected upload is still drained, and the 2000 ms monitoring interval. `session_t` holds everything that belongs to a single connection: the socket and peer address, the authentication flag, the receive buffer with its read position and fill level (used for framing), and the state of the connection's monitor thread (`mon_active`, `mon_port`, `mon_thread`, `atomic mon_stop`).

5.2 Connection handling and concurrency

```
int main(void)
{
    signal(SIGPIPE, SIG_IGN);
    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sa.sa_handler = on_signal;          /* no SA_RESTART: accept() wakes */
    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);
    srand((unsigned)time(NULL));

    g_log = fopen(LOG_FILE, "a");
    if (!g_log) { perror("open log"); return 1; }
    mkdir(STORE_ROOT, 0755);
    mkdir(STORE_DIR, 0755);

    int ls = socket(AF_INET, SOCK_STREAM, 0);
    if (ls < 0) { perror("socket"); return 1; }
    int yes = 1;
    setsockopt(ls, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);

    struct sockaddr_in addr;
    memset(&addr, 0, sizeof addr);
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(LISTEN_PORT);
    if (bind(ls, (struct sockaddr *)&addr, sizeof addr) < 0) {
        perror("bind"); return 1;
    }
    if (listen(ls, 16) < 0) { perror("listen"); return 1; }

    log_msg("RemoteOps Agent (%s) listening on TCP port %d, SID:%s, "
            "storage %s", REG_NO, LISTEN_PORT, SID, STORE_DIR);

    while (g_running) {
        struct sockaddr_in cli;
        socklen_t cl = sizeof cli;
    }
}

int main(void)
{
    signal(SIGPIPE, SIG_IGN);
    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sa.sa_handler = on_signal;          /* no SA_RESTART: accept() wakes */
    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);
    srand((unsigned)time(NULL));

    g_log = fopen(LOG_FILE, "a");
    if (!g_log) { perror("open log"); return 1; }
    mkdir(STORE_ROOT, 0755);
    mkdir(STORE_DIR, 0755);

    int ls = socket(AF_INET, SOCK_STREAM, 0);
    if (ls < 0) { perror("socket"); return 1; }
    int yes = 1;
    setsockopt(ls, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);

    struct sockaddr_in addr;
    memset(&addr, 0, sizeof addr);
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons(LISTEN_PORT);
    if (bind(ls, (struct sockaddr *)&addr, sizeof addr) < 0) {
        perror("bind"); return 1;
    }
    if (listen(ls, 16) < 0) { perror("listen"); return 1; }

    log_msg("RemoteOps Agent (%s) listening on TCP port %d, SID:%s, "
            "storage %s", REG_NO, LISTEN_PORT, SID, STORE_DIR);

    while (g_running) {
        struct sockaddr_in cli;
        socklen_t cl = sizeof cli;
    }
}
```

Figure 5.2: *main()* accept loop and *client_thread()*

client_thread() (top) is the per-connection handler: it logs the connection, then loops *read_line()* → *handle_line()* until the Controller quits or disconnects. After the loop it stops the monitor thread, closes the socket, logs the disconnect (clean QUIT or connection lost) and frees the session. In *main()* (bottom), the accept loop creates a *session_t* for each accepted socket, records the peer address, and starts a **detached** thread, so finished threads clean up themselves. If thread creation fails, the connection is closed and the session freed, and the Agent keeps serving others.

5.3 Line framing and byte counting

```
/* Returns length, 0 on success, size_t on error, 0 on line too long. */
static int read_line(session_t *s, char *out, size_t max)
{
    size_t o = 0;
    for (;;) {
        while (s->rpos < s->rlen) {
            char c = s->rbuf[s->rpos++];
            if (c == '\n') {
                out[o] = '\0';
                if (o > 0 && out[o - 1] == '\r') out[--o] = '\0';
                return (int)o;
            }
            if (o + 1 >= max) return -2;
            out[o++] = c;
        }
        ssize_t k = recv(s->fd, s->rbuf, sizeof s->rbuf, 0);
        if (k < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        if (k == 0) return -1;
        s->rpos = 0;
        s->rlen = (size_t)k;
    }
}

/* Read up to n raw bytes; uses bytes already buffered by read_line first. */
static ssize_t read_some(session_t *s, void *buf, size_t n)
{
    if (s->rpos < s->rlen) {
        size_t avail = s->rlen - s->rpos;
        if (avail > n) avail = n;
        memcpy(buf, s->rbuf + s->rpos, avail);
        s->rpos += avail;
        return (ssize_t)avail;
    }
    for (;;) {
        ssize_t k = recv(s->fd, buf, n, 0);
        if (k < 0 && errno == EINTR) continue;
        return k;
    }
}
```

Figure 5.3: `read_line()` and `read_some()`

`read_line()` consumes characters from the session buffer until `\n`; when the buffer is empty it refills it with one `recv()`. A return value of 0 or `-1` means the Controller closed or reset the connection, which ends the session. `read_some()` is used for raw file bytes: it first hands out whatever is still in the buffer after the header line, then reads directly from the socket. Together they guarantee that no byte is lost or read twice between text lines and file data.

5.4 Authentication and command dispatch

```
static int read_line(session_t *s, char *out, size_t max)
{
    size_t o = 0;
    for (;;) {
        while (s->rpos < s->rlen) {
            char c = s->rbuf[s->rpos++];
            if (c == '\n') {
                out[o] = '\0';
                if (o > 0 && out[o - 1] == '\r') out[--o] = '\0';
                return (int)o;
            }
            if (o + 1 >= max) return -2;
            out[o++] = c;
        }
        ssize_t k = recv(s->fd, s->rbuf, sizeof s->rbuf, 0);
        if (k < 0) {
            if (errno == EINTR) continue;
            return -1;
        }
        if (k == 0) return -1;
        s->rpos = 0;
        s->rlen = (size_t)k;
    }
}

/* Read up to n raw bytes; uses bytes already buffered by read_line first. */
static ssize_t read_some(session_t *s, void *buf, size_t n)
{
    if (s->rpos < s->rlen) {
        size_t avail = s->rlen - s->rpos;
        if (avail > n) avail = n;
        memcpy(buf, s->rbuf + s->rpos, avail);
        s->rpos += avail;
        return (ssize_t)avail;
    }
    for (;;) {
        ssize_t k = recv(s->fd, buf, n, 0);
        if (k < 0 && errno == EINTR) continue;
        return k;
    }
}

const char *real = a1 ? whitelist_lookup(a1) : NULL;
if (!real) {
    log_msg("%s EXEC '%s' rejected (not whitelisted)", s->peer_str,
            a1 ? a1 : "");
    return reply(s, "ERR 002 COMMAND_NOT_ALLOWED") < 0;
}
char out[1024];
run_fixed(real, out, sizeof out);
return reply(s, "OK EXEC_RESULT %s", out) < 0;
}

if (strcmp(cmd, "PUT") == 0) return do_put(s, a1, a2);
if (strcmp(cmd, "GET") == 0) return do_get(s, a1);

if (strcmp(cmd, "MONITOR") == 0) {
    if (a1 && strcmp(a1, "START") == 0) {
        char *e;
        long port = a2 ? strtol(a2, &e, 10) : 0;
        if (!a2 || *e != '\0' || port < 1 || port > 65535)
            return reply(s, "ERR 006 BAD_REQUEST") < 0;
        if (s->mon_active)
            return reply(s, "ERR 007 MONITOR_ALREADY_RUNNING") < 0;
        s->mon_port = (int)port;
        atomic_store(&s->mon_stop, 0);
        if (pthread_create(&s->mon_thread, NULL, monitor_thread, s) != 0)
            return reply(s, "ERR 012 INTERNAL_ERROR") < 0;
        s->mon_active = 1;
        log_msg("%s MONITOR stream started -> udp port %ld (every %d ms)",
                s->peer_str, port, MONITOR_INTERVAL_MS);
        return reply(s, "OK MONITOR_STARTED") < 0;
    }
    if (a1 && strcmp(a1, "STOP") == 0) {
        stop_monitor(s);
        return reply(s, "OK MONITOR_STOPPED") < 0;
    }
    return reply(s, "ERR 006 BAD_REQUEST") < 0;
}

if (strcmp(cmd, "QUIT") == 0) {
    stop_monitor(s);
    reply(s, "OK BYE");
    s->quit_clean = 1;
    return 1;
}
return reply(s, "ERR 009 UNKNOWN_COMMAND") < 0;
}
```

Figure 5.4: `handle_line()` authentication and dispatch

handle_line() splits the line into a command and up to two arguments with strtok_r(). It logs the line (replacing the token with <token> for AUTH), then checks AUTH first. For every other command the line if (!s->authed) returns ERR 008 NOT_AUTHENTICATED, so nothing can be executed before a successful AUTH OPS-3381. The remaining commands are dispatched in turn, and unknown commands fall through to ERR 009. Every reply goes through reply(), which appends SID:1833 and the newline in one place, so no response can miss the tag.

5.5 SYSINFO and LISTPROC

```

/*
static void get_sysinfo(char *out, size_t n)
{
    double load = -1.0, up = -1.0;
    long total = 0, avail = -1;
    FILE *f;

    if ((f = fopen("/proc/loadavg", "r"))) {
        if (fscanf(f, "%lf", &load) != 1) load = -1.0;
        fclose(f);
    }
    if ((f = fopen("/proc/meminfo", "r"))) {
        char line[256];
        while (fgets(line, sizeof line, f)) {
            sscanf(line, "MemTotal: %ld", &total);
            sscanf(line, "MemAvailable: %ld", &avail);
        }
        fclose(f);
    }
    if ((f = fopen("/proc/uptime", "r"))) {
        if (fscanf(f, "%lf", &up) != 1) up = -1.0;
        fclose(f);
    }
    long mem_mb;
    if (avail >= 0 && total > 0) mem_mb = (total - avail) / 1024;
    else mem_mb = 512 + rand() % 256; /* simulated */
    if (load < 0) load = 0.10 + (rand() % 50) / 100.0; /* simulated */
    if (up < 0) up = (double)(time(NULL) % 100000); /* simulated */

    snprintf(out, n, "SYSINFO %.2f %ld %ld", load, mem_mb, (long)up);
}

/* Process snapshot straight from /proc: "pid:name,pid:name,..." */
static void get_procs(char *out, size_t n)
{
    DIR *d = opendir("/proc");
    size_t used = 0;
    out[0] = '\0';
    if (!d) { snprintf(out, n, "unavailable"); return; }
    struct dirent *e;
    while ((e = readdir(d))) {
        if (!isdigit((unsigned char)e->d_name[0])) continue;
        char path[300], name[64] = "?";
        snprintf(path, sizeof path, "/proc/%s/comm", e->d_name);
        FILE *f = fopen(path, "r");
        if (f) {
            if (fgets(name, sizeof name, f)) name[strcspn(name, "\n")] = '\0';
            fclose(f);
        }
        for (char *p = name; *p; p++)
            if (*p == ' ' || *p == ',') *p = '_';
        int w = snprintf(out + used, n - used, "%s%s:%s",
            used ? ", " : "", e->d_name, name);
        if (w < 0 || (size_t)w >= n - used) break; /* buffer full */
        used += (size_t)w;
    }
    closedir(d);
    if (used == 0) snprintf(out, n, "none");
}

```

Figure 5.5: get_sysinfo() and get_procs()

get_sysinfo() reads real values from Linux: the 1-minute load average from /proc/loadavg, used memory as MemTotal - MemAvailable from /proc/meminfo (in MB), and uptime in seconds from /proc/uptime. If a file cannot be read, a plausible simulated value is used, as the brief allows. get_procs() builds the process snapshot by scanning the numeric directories of /proc and reading each process name from its comm file, producing pid:name,pid:name,... and stopping safely when the output buffer is full.

5.6 Restricted EXEC

```

/* ----- */
/* EXEC: strict whitelist. The client text is never put in a command. */
/* ----- */
static const struct { const char *name; const char *cmd; } WHITELIST[] = {
    { "DATE", "date" },
    { "UPTIME", "uptime -p" },
    { "DISKFREE", "df -h /" },
    { "HOSTNAME", "hostname" },
    { "WHOAMI", "whoami" },
};

static const char *whitelist_lookup(const char *name)
{
    for (size_t i = 0; i < sizeof WHITELIST / sizeof WHITELIST[0]; i++)
        if (strcmp(name, WHITELIST[i].name) == 0) return WHITELIST[i].cmd;
    return NULL;
}

static void run_fixed(const char *cmd, char *out, size_t n)
{
    FILE *p = popen(cmd, "r");
    size_t used = 0;
    out[0] = '\0';
    if (!p) { snprintf(out, n, "exec_failed"); return; }
    int c;
    while ((c = fgetc(p)) != EOF && used + 1 < n)
        out[used++] = (c == '\n') ? '|' : (char)c; /* keep one-line rule */
    out[used] = '\0';
    pclose(p);
    while (used > 0 && out[used - 1] == '|') out[--used] = '\0';
    if (used == 0) snprintf(out, n, "(no output)");
}

```

Figure 5.6: WHITELIST table and run_fixed()

EXEC is implemented with a constant table that maps each allowed name (DATE, UPTIME, DISKFREE, HOSTNAME, WHOAMI) to a fixed command string. whitelist_lookup() compares the received name to the table with strcmp(); only on an exact match is the table's own command passed to popen(). Text sent by the Controller is never placed in a command line, so shell injection is impossible. run_fixed() collects the output and replaces newlines with | so the response stays on one line.

5.7 File upload and download

```
/* ----- */
/* File transfer */
/* ----- */
/* Only simple names: letters, digits, '.', '_', '-', no leading dot. */
static int valid_filename(const char *f)
{
    size_t l = strlen(f);
    if (l == 0 || l > 100 || f[0] == '.') return 0;
    for (; *f; f++) {
        unsigned char c = (unsigned char)*f;
        if (!(isalnum(c) || c == '.' || c == '_' || c == '-')) return 0;
    }
    return 1;
}

static int do_put(session_t *s, const char *fname, const char *sizestr)
{
    if (!fname || !sizestr || sizestr[0] == '-' || sizestr[0] == '\0') {
        reply(s, "ERR 006 BAD_REQUEST");
        return 0;
    }
    char *end;
    errno = 0;
    unsigned long long sz = strtoull(sizestr, &end, 10);
    if (*end != '\0' || errno != 0) {
        reply(s, "ERR 006 BAD_REQUEST");
        return 0;
    }

    if (sz > MAX_FILE_SIZE) {
        log_msg("%s PUT %s rejected: %llu bytes exceeds limit",
            s->peer_str, fname, sz);
        if (sz > DRAIN_LIMIT) { reply(s, "ERR 004 FILE_TOO_LARGE"); return 1; }
        if (drain(s, sz) < 0) return 1;
        reply(s, "ERR 004 FILE_TOO_LARGE");
        return 0;
    }
    if (!valid_filename(fname)) {
        if (drain(s, sz) < 0) return 1; /* keep framing intact */
        reply(s, "ERR 003 BAD_FILENAME");
        return 0;
    }

    char final_path[256], tmp_path[300];
    snprintf(final_path, sizeof final_path, "%s/%s", STORE_DIR, fname);
    snprintf(tmp_path, sizeof tmp_path, "%s/.%s.part.%lu",
        STORE_DIR, fname, (unsigned long)pthread_self());
    FILE *f = fopen(tmp_path, "wb");
    if (!f) {
        if (drain(s, sz) < 0) return 1;
        reply(s, "ERR 011 STORAGE_ERROR");
        return 0;
    }

    unsigned long long remaining = sz;
    char buf[8192];
    while (remaining > 0) {
        size_t want = remaining > sizeof buf ? sizeof buf : (size_t)remaining;
        ssize_t k = read_some(s, buf, want);

        unsigned long long remaining = sz;
        char buf[8192];
        while (remaining > 0) {
            size_t want = remaining > sizeof buf ? sizeof buf : (size_t)remaining;
            ssize_t k = read_some(s, buf, want);
            if (k <= 0) { /* client vanished mid-upload */
                fclose(f);
                unlink(tmp_path);
                log_msg("%s PUT %s aborted (connection lost, %llu bytes missing)",
                    s->peer_str, fname, remaining);
                return 1;
            }
            fwrite(buf, 1, (size_t)k, f);
            remaining -= (unsigned long long)k;
        }
        fclose(f);
        if (rename(tmp_path, final_path) != 0) {
            unlink(tmp_path);
            reply(s, "ERR 011 STORAGE_ERROR");
            return 0;
        }
        log_msg("%s PUT %s %llu bytes stored at %s", s->peer_str, fname, sz,
            final_path);
        return reply(s, "OK FILE_RECEIVED %s", fname) < 0;
    }
}
```

```

static int do_get(session_t *s, const char *fname)
{
    if (!fname || !valid_filename(fname)) {
        reply(s, "ERR 005 FILE_NOT_FOUND");
        return 0;
    }
    char path[256];
    snprintf(path, sizeof path, "%s/%s", STORE_DIR, fname);
    struct stat st;
    FILE *f = fopen(path, "rb");
    if (!f || fstat(fileno(f), &st) != 0 || !S_ISREG(st.st_mode)) {
        if (f) fclose(f);
        log_msg("%s GET %s: not found", s->peer_str, fname);
        reply(s, "ERR 005 FILE_NOT_FOUND");
        return 0;
    }
    unsigned long long size = (unsigned long long)st.st_size;
    if (reply(s, "OK FILE_SEND %s %llu", fname, size) < 0) {
        fclose(f);
        return 1;
    }
    char buf[8192];
    unsigned long long remaining = size;
    while (remaining > 0) {
        size_t want = remaining > sizeof buf ? sizeof buf : (size_t)remaining;
        size_t k = fread(buf, 1, want, f);
        if (k == 0 || send_all(s->fd, buf, k) < 0) {
            fclose(f);
            log_msg("%s GET %s aborted", s->peer_str, fname);
            return 1; /* promised bytes can't be honoured */
        }
        remaining -= k;
    }
    fclose(f);
    log_msg("%s GET %s %llu bytes sent", s->peer_str, fname, size);
    return 0;
}

```

Figure 5.7: `valid_filename()`, `do_put()` and `do_get()`

`valid_filename()` accepts only letters, digits, `.`, `_` and `-`, at most 100 characters, with no leading dot, so path tricks such as `../x` or `/etc/passwd` are rejected. `do_put()` parses the size with `strtoull()`; a file above 10 MB gets `ERR 004` after its bytes are drained, and a bad name gets `ERR 003` after draining. Valid uploads are written to a temporary file and renamed when exactly `<filesize>` bytes have arrived. If the Controller disappears mid-upload, the temporary file is deleted and the session ends. `do_get()` checks that the file exists and is a regular file (otherwise `ERR 005`), sends `OK FILE_SEND <name> <size>`, then sends exactly that many bytes with `send_all()`.

5.8 UDP monitoring loop

```

/* ----- */
/* UDP monitoring */
/* ----- */
static void *monitor_thread(void *arg)
{
    session_t *s = arg;
    int u = socket(AF_INET, SOCK_DGRAM, 0);
    if (u < 0) return NULL;
    struct sockaddr_in dst = s->peer;
    dst.sin_port = htons((uint16_t)s->mon_port);

    while (!atomic_load(&s->mon_stop)) {
        char info[128], msg[192];
        get_sysinfo(info, sizeof info);
        snprintf(msg, sizeof msg, "%s SID:%s", info, SID);
        sendto(u, msg, strlen(msg), 0, (struct sockaddr *)&dst, sizeof dst);
        for (int i = 0; i < MONITOR_INTERVAL_MS / 100 &&
             !atomic_load(&s->mon_stop); i++)
            usleep(100000);
    }
    close(u);
    return NULL;
}

static void stop_monitor(session_t *s)
{
    if (s->mon_active) {
        atomic_store(&s->mon_stop, 1);
        pthread_join(s->mon_thread, NULL);
        s->mon_active = 0;
        log_msg("%s MONITOR stream stopped", s->peer_str);
    }
}

```

Figure 5.8: `monitor_thread()` and `stop_monitor()`

MONITOR START <udp_port> starts one monitor_thread() for the session. It opens its own UDP socket, takes the Controller's IP address from the TCP connection (s->peer) and the port from the command, and every 2 seconds sends SYSINFO <cpu_load> <mem_used_mb> <uptime_sec> SID:1833. The sleep is split into 100 ms steps that check the atomic mon_stop flag, so a stop request takes effect within a tenth of a second. stop_monitor() sets the flag and joins the thread; it is called by MONITOR STOP, by QUIT, and by the cleanup after a lost connection.

5.9 Logging

```
/* ----- */
/* Logging: every line timestamped, written to file and stdout */
/* ----- */
static void log_msg(const char *fmt, ...)
{
    char ts[32];
    time_t t = time(NULL);
    struct tm tmv;
    localtime_r(&t, &tmv);
    strftime(ts, sizeof ts, "%Y-%m-%d %H:%M:%S", &tmv);

    pthread_mutex_lock(&g_log_mu);
    va_list ap;
    if (g_log) {
        va_start(ap, fmt);
        fprintf(g_log, "[%s] ", ts);
        vfprintf(g_log, fmt, ap);
        fputc('\n', g_log);
        fflush(g_log);
        va_end(ap);
    }
    va_start(ap, fmt);
    printf("[%s] ", ts);
    vprintf(fmt, ap);
    putchar('\n');
    fflush(stdout);
    va_end(ap);
    pthread_mutex_unlock(&g_log_mu);
}
```

Figure 5.9: log_msg()

log_msg() formats a timestamp with localtime_r() and strftime(), then, holding the mutex, writes the line to both the log file and the terminal and flushes the file immediately, so the log stays complete even if the Agent is stopped. Connections, authentication results, every command, file transfers, monitor start/stop and disconnects all go through this one function.

5.10 Controller

```

/* ----- UDP monitor receiver ----- */
static void *udp_thread(void *arg)
{
    (void)arg;
    char buf[512];
    while (!atomic_load(&g_ustop)) {
        ssize_t k = recvfrom(g_usock, buf, sizeof buf - 1, 0, NULL, NULL);
        if (k > 0) {
            buf[k] = '\0';
            printf("\n[UDP monitor] %s\n", buf);
            fflush(stdout);
        }
    }
    return NULL;
}

static int start_udp_listener(int port)
{
    if (g_urunning) return 0;
    g_usock = socket(AF_INET, SOCK_DGRAM, 0);
    if (g_usock < 0) return -1;
    int yes = 1;
    setsockopt(g_usock, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
    struct timeval tv = { 0, 500000 }; /* wake every 0.5 s */
    setsockopt(g_usock, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
    struct sockaddr_in a;
    memset(&a, 0, sizeof a);
    a.sin_family = AF_INET;
    a.sin_addr.s_addr = htonl(INADDR_ANY);
    a.sin_port = htons((uint16_t)port);
    if (bind(g_usock, (struct sockaddr *)&a, sizeof a) < 0) {
        perror("udp bind");
        close(g_usock);
        g_usock = -1;
        return -1;
    }

    a.sin_family = AF_INET;
    a.sin_addr.s_addr = htonl(INADDR_ANY);
    a.sin_port = htons((uint16_t)port);
    if (bind(g_usock, (struct sockaddr *)&a, sizeof a) < 0) {
        perror("udp bind");
        close(g_usock);
        g_usock = -1;
        return -1;
    }
    atomic_store(&g_ustop, 0);
    if (pthread_create(&g_uthread, NULL, udp_thread, NULL) != 0) {
        close(g_usock);
        g_usock = -1;
        return -1;
    }
    g_urunning = 1;
    return 0;
}

```

```

static void do_put(const char *path)
{
    if (!path) { puts("usage: PUT <local_file>"); return; }
    FILE *f = fopen(path, "rb");
    struct stat st;
    if (!f || stat(path, &st) != 0) { perror(path); if (f) fclose(f); return; }
    const char *base = strrchr(path, '/');
    base = base ? base + 1 : path;
    unsigned long long size = (unsigned long long)st.st_size;

    char hdr[512];
    snprintf(hdr, sizeof hdr, "PUT %s %llu\n", base, size);
    double t0 = now_sec();
    if (send_all(hdr, strlen(hdr)) < 0) lost();
    char buf[8192];
    size_t k;
    while ((k = fread(buf, 1, sizeof buf, f)) > 0)
        if (send_all(buf, k) < 0) lost();
    fclose(f);

    char resp[LINE_MAX_LEN];
    if (read_line(resp, sizeof resp) < 0) lost();
    double dt = now_sec() - t0;
    printf("%s\n", resp);
    if (strcmp(resp, "OK", 2) == 0 && dt > 0)
        printf("[PUT] %llu bytes in %.4f s = %.0f bytes/s\n", size, dt,
            size / dt);
}

```

Figure 5.10: UDP receiver and do_put() (controller_381.c)

The Controller keeps its own receive buffer in the same way as the Agent. `do_put()` reads the local file, sends `PUT <basename> <size>` followed by the raw bytes, waits for the reply, and prints the throughput measured with `clock_gettime(CLOCK_MONOTONIC)`; `do_get()` does the reverse. For `MONITOR START <port>` the Controller first binds a UDP socket on that port and starts `udp_thread()`, which prints each datagram as `[UDP monitor] ...`, and only then sends the command to the Agent. All other commands are sent exactly as typed, so error cases can be tested directly.

6. Execution and Output Screenshots

All screenshots in this section come from my own runs of the Agent and Controller. The Agent was started with `./agent_381` and the Controller with `./controller_381`.

6.1 Agent starting up

The Agent prints and logs its personalised settings when it starts: TCP port 9410, SID 1833 and the storage path.

```
manu@localhost:~/IE3090_RemoteOps_IT24103381$ ./agent_381
[2026-10-07 01:54:46] RemoteOps Agent (IT24103381) listening on TCP port 9410, SID:1833, storage ./agentfiles/IT24103381
|
```

Figure 6.1: Agent starting up

6.2 Successful AUTH

A Controller must authenticate first. A command before AUTH is rejected with `ERR 008`, and the correct token `OPS-3381` returns `OK AUTHENTICATED SID:1833`.

```
Last login: Wed Oct  7 02:06:23 2026 from 172.20.10.4
manu@localhost:~$ cd ~/IE3090_RemoteOps_IT24103381
manu@localhost:~/IE3090_RemoteOps_IT24103381$ ./controller_381 127.0.0.1
Connected to 127.0.0.1:9410. Start with: AUTH <token>
remoteops> AUTH OPS-3381
OK AUTHENTICATED SID:1833
remoteops> SYSINFO
```

Figure 6.2: Successful AUTH

6.3 SYSINFO, LISTPROC and EXEC

`SYSINFO` and `LISTPROC` return live data from `/proc`. `EXEC DATE` and `EXEC HOSTNAME` run the fixed commands and return their output in `OK EXEC_RESULT` lines. The captured `EXEC` exchange is shown below.

```

remoteops> SYSINFO
OK SYSINFO 0.04 1203 6523 SID:1833
remoteops> LISTPROC
OK PROC 1:systemd,2:kthreadd,3:pool_workqueue_release,4:kworker/R-rcu_gp,5:kworker/R-sync_wq,6:kworker/R-kvfree_r
cu_reclaim,7:kworker/R-slub_flushwq,8:kworker/R-netns,10:kworker/0:0H-events_highpri,11:kworker/0:1-mpt_poll,12:
kworker/u512:0-events_unbound,13:kworker/R-mm_percpu_wq,14:kworker/u512:1-ipv6_addrconf,15:ksoftirqd/0,16:rcu_pree
mpt,17:rcu_exp_par_gp_kthread_worker/1,18:rcu_exp_gp_kthread_worker,19:migration/0,20:idle_inject/0,21:cpuhp/0,22:
cpuhp/1,23:idle_inject/1,24:migration/1,25:ksoftirqd/1,27:kworker/1:0H-events_highpri,28:cpuhp/2,29:idle_inject/2,
30:migration/2,31:ksoftirqd/2,33:kworker/2:0H-events_highpri,34:cpuhp/3,35:idle_inject/3,36:migration/3,37:ksoftir
qd/3,39:kworker/3:0H-kblockd,41:kworker/u514:0-kvfree_rcu_reclaim,43:kworker/u514:1-events_unbound,44:kworker/u514
:2-events_unbound,46:kdevtmpfs,47:kworker/R-inet_frag_wq,48:rcu_tasks_kthread,49:rcu_tasks_rude_kthread,50:rcu_tas
ks_trace_kthread,52:kauditd,53:khungtaskd,54:oom_reaper,56:kworker/R-writeback,57:kcompactd0,58:ksmd,59:khugepaged
,60:kworker/R-cryptd,61:kworker/R-kblockd,62:kworker/R-blkcq_punt_bio,63:kworker/R-kintegrityd,64:irq/9-acpi,67:kw
orker/R-tpm_dev_wq,68:kworker/R-md_bitmap,69:kworker/R-edac_poller,71:watchdogd,74:kswapd0,91:kwor
ker/R-kthrotld,96:irq/24-pciehp,97:irq/25-pciehp,98:irq/26-pciehp,99:irq/27-pciehp,100:irq/28-pciehp,101:irq/29-pc
iehp,102:irq/30-pciehp,103:irq/31-pciehp,104:irq/32-pciehp,105:irq/33-pciehp,106:irq/34-pciehp,107:irq/35-pciehp,1
08:irq/36-pciehp,109:irq/37-pciehp,110:irq/38-pciehp,111:irq/39-pciehp,112:irq/40-pciehp,113:irq/41-pciehp,114:irq
/42-pciehp,115:irq/43-pciehp,116:irq/44-pciehp,117:irq/45-pciehp,118:irq/46-pciehp,119:irq/47-pciehp,120:irq/48-pc
iehp,121:irq/49-pciehp,122:irq/50-pciehp,123:irq/51-pciehp,124:irq/52-pciehp,125:irq/53-pciehp,126:irq/54-pciehp,1
27:irq/55-pciehp,128:kworker/R-acpi_thermal_pm,129:kworker/R-kmpath_rdacd,130:kworker/R-kalua,132:kworker/R-mld,1
34:kworker/R-ipv6_addrconf,139:kworker/R-kstrp,145:kworker/u515:0,146:kworker/u516:0-ttm,147:kworker/u517:0,198:kw
orker/u513:6-events_unbound,289:kworker/3:1H-kblockd,331:kworker/2:2-events,339:kworker/R-kmpathd,340:kworker/R-km
path_handlerd,479:kworker/R-iscsi_conn_cleanup,483:kworker/R-tls-strp,485:kworker/R-cnic_wq,486:bnx2i_thread/0,487
:bnx2i_thread/1,488:bnx2i_thread/2,489:bnx2i_thread/3,653:kworker/R-nvme-wq,654:kworker/R-nvme-reset-wq,655:kworke
r/R-nvme-delete-wq,656:kworker/R-nvme-auth-wq,658:kworker/R-nvme-tcp_wq,659:kworker/R-ata_sff,666:kworker/R-mpt_po
ll,667:kworker/R-mpt/0,678:scsi_ah_0,679:kworker/R-scsi_tmf_0,680:scsi_ah_1,682:kworker/R-scsi_tmf_1,686:scsi_ah
_2,687:kworker/R-scsi_tmf_2,771:kworker/R-kdmflush/253:0,778:kworker/R-kdmflush/253:1,810:kworker/R-xfsalloc,811:k
worker/R-xfs_mru_cache,812:kworker/R-xfs-buf/dm-0,813:kworker/R-xfs-conv/dm-0,814:kworker/R-xfs-reclaim/dm-0,815:k
worker/R-xfs-blockgc/dm-0,816:kworker/R-xfs-inodegc/dm-0,817:kworker/R-xfs-log/dm-0,818:kworker/R-xfs-cil/dm-0,819
:xfsaild/dm-0,894:irq/56-vmm_vhci,895:irq/57-vmm_vhci,896:irq/58-vmm_vhci,912:systemd-journal,957:systemd-udev,10
06:kworker/R-xfs-buf/sda2,1007:kworker/R-xfs-conv/sda2,1008:kworker/R-xfs-reclaim/sda2,1009:kworker/R-xfs-blockgc/
sda2,1010:kworker/R-xfs-inodegc/sda2,1011:kworker/R-xfs-log/sda2,1012:kworker/R-xfs-cil/sda2,1013:xfsaild/sda2,102
9:auditd,1031:sedispacth,1044:irq/16-vmmgfx,1045:kworker/R-ttm,1057:dbus-broker-lau,1092:dbus-broker,1098:avahi-da
emon,1099:chronyd,1101:firewalld,1102:NetworkManager,1105:irqbalance,1106:lsmd,1107:mcelog,1108:polkitd,1109:rtkit
-daemon,1110:accounts-daemon,1111:switcheeroo-cont,1114:systemd-logind,1115:udisksd,1117:upowerd,1119:VGAuthService
,1124:vmtoolsd,1129:systemd-userdbd,1164:avahi-daemon,1170:alsactl,1208:ModemManager,1218:cupsd,1219:sshd,1222:tun
ed,1250:atd,1251:crond,1254:gdm,1338:tuned-ppd,1390:rsyslogd,2059:colord,2698:wpa_supplicant,2896:kworker/u516:2-t
tm,2927:gdm-session-wor,2940:systemd,2942:(sd-pan),2961:gnome-keyring-d,2971:gdm-wayland-ses,2977:dbus-broker-lau,
2984:dbus-broker,2989:gnome-session-b,3039:gnome-session-c,3043:gnome-session-b,3071:gnome-shell,3078:gvfsd,3084:g
SID:1833
remoteops> EXEC DATE
OK EXEC_RESULT Wed Oct 7 02:08:08 AM EDT 2026 SID:1833
remoteops> EXEC HOSTNAME
OK EXEC_RESULT localhost.localdomain SID:1833

```

Figure 6.3: SYSINFO, LISTPROC and EXEC

6.4 Successful PUT and GET

PUT testfile.txt uploads a 16-byte file; the Agent stores it and answers OK FILE_RECEIVED. GET testfile.txt returns OK FILE_SEND testfile.txt 16 followed by the 16 bytes, which the Controller saves as dl_testfile.txt. The Controller prints the throughput of each transfer (383 bytes/s for the PUT and 772 bytes/s for the GET in this small test).

```

OK EXEC_RESULT localhost.localdomain SID:1833
remoteops> PUT testfile.txt
OK FILE_RECEIVED testfile.txt SID:1833
[PUT] 37 bytes in 0.0477 s = 775 bytes/s
remoteops> GET testfile.txt
OK FILE_SEND testfile.txt 37 SID:1833
[GET] saved 37 bytes to dl_testfile.txt in 0.0015 s = 24965 bytes/s

```

Figure 6.4: Successful PUT and GET

6.5 UDP monitoring stream

After MONITOR START 5000 the Agent sends a SYSINFO ... SID:1833 datagram every two seconds to UDP port 5000 of the Controller, which prints each one as [UDP monitor] ... MONITOR STOP stops the stream.

```

remoteops> MONITOR START 9500
OK MONITOR_STARTED SID:1833
remoteops>
[UDP monitor] SYSINFO 0.00 1191 6751 SID:1833

[UDP monitor] SYSINFO 0.00 1191 6753 SID:1833

[UDP monitor] SYSINFO 0.00 1191 6755 SID:1833

[UDP monitor] SYSINFO 0.08 1191 6757 SID:1833

[UDP monitor] SYSINFO 0.08 1191 6759 SID:1833

[UDP monitor] SYSINFO 0.08 1191 6761 SID:1833

[UDP monitor] SYSINFO 0.07 1191 6763 SID:1833

[UDP monitor] SYSINFO 0.07 1191 6765 SID:1833
MONITOR STOP
OK MONITOR_STOPPED SID:1833

```

Figure 6.5: UDP monitoring stream

6.6 Error cases handled gracefully

Invalid input is answered with a specific error code and the connection stays usable: a wrong token (ERR 001), a command before authentication (ERR 008), a command that is not on the whitelist (ERR 002) and a file that does not exist (ERR 005).

```

manu@localhost:~/IE3090_RemoteOps_IT24103381$ ./controller_381 127.0.0.1
Connected to 127.0.0.1:9410. Start with: AUTH <token>
remoteops> SYSINFO
ERR 008 NOT_AUTHENTICATED SID:1833
remoteops> AUTH WRONGTOKEN
ERR 001 AUTH_FAILED SID:1833
remoteops> AUTH OPS-3381
OK AUTHENTICATED SID:1833
remoteops> EXEC rm -rf /
ERR 002 COMMAND_NOT_ALLOWED SID:1833
remoteops> GET nowhere.txt
ERR 005 FILE_NOT_FOUND SID:1833
remoteops> |

```

Figure 6.6: Error cases handled gracefully

6.7 Five simultaneous Controllers

Five Controllers were connected to one Agent at the same time, each authenticated and each receiving correct replies, which shows that the thread-per-connection design serves several clients concurrently.

```

Last login: wed Oct  7 02:58:52 2026 from 172.20.10.4
manu@localhost:~$ cd ~/IE3090_RemoteOps_IT24103381
manu@localhost:~/IE3090_RemoteOps_IT24103381$ for i in 1 2 3 4 5; do (echo "AUTH OPS-3381"; sleep 1; echo "SYSINFO"; sleep 1; echo "QUIT") | ./controller_381 127.0.0.1 > /tmp/ctrl_${i}.log 2>&1 & done
[1] 5922
[2] 5925
[3] 5928
[4] 5931
[5] 5933
manu@localhost:~/IE3090_RemoteOps_IT24103381$ sleep 6
[1] Done ( echo "AUTH OPS-3381"; sleep 1; echo "SYSINFO"; sleep 1; echo "QUIT" )
| ./controller_381 127.0.0.1 > /tmp/ctrl_${i}.log 2>&1
[2] Done ( echo "AUTH OPS-3381"; sleep 1; echo "SYSINFO"; sleep 1; echo "QUIT" )
| ./controller_381 127.0.0.1 > /tmp/ctrl_${i}.log 2>&1
[3] Done ( echo "AUTH OPS-3381"; sleep 1; echo "SYSINFO"; sleep 1; echo "QUIT" )
| ./controller_381 127.0.0.1 > /tmp/ctrl_${i}.log 2>&1
[4]- Done ( echo "AUTH OPS-3381"; sleep 1; echo "SYSINFO"; sleep 1; echo "QUIT" )
| ./controller_381 127.0.0.1 > /tmp/ctrl_${i}.log 2>&1
[5]+ Done ( echo "AUTH OPS-3381"; sleep 1; echo "SYSINFO"; sleep 1; echo "QUIT" )
| ./controller_381 127.0.0.1 > /tmp/ctrl_${i}.log 2>&1
manu@localhost:~/IE3090_RemoteOps_IT24103381$ grep CONNECTED remoteops_IT24103381.log | tail -10
manu@localhost:~/IE3090_RemoteOps_IT24103381$ head -50 remoteops_IT24103381.log
[2026-10-06 09:16:03] RemoteOps Agent (IT24103381) listening on TCP port 9410, SID:1833, storage ./agentfiles/IT24103381
[2026-10-06 09:16:44] 127.0.0.1:44950 connected
[2026-10-06 09:18:31] 127.0.0.1:44950 CMD AUTH <token>
[2026-10-06 09:18:31] 127.0.0.1:44950 authentication OK
[2026-10-06 09:18:42] 127.0.0.1:44950 CMD SYSINFO
[2026-10-06 09:18:58] 127.0.0.1:44950 CMD LISTPROC
[2026-10-06 09:19:12] 127.0.0.1:44950 CMD EXEC DATE
[2026-10-06 09:19:26] 127.0.0.1:44950 CMD EXEC HOSTNAME
[2026-10-06 09:19:40] 127.0.0.1:44950 CMD PUT testfile.txt 16
[2026-10-06 09:19:40] 127.0.0.1:44950 PUT testfile.txt 16 bytes stored at ./agentfiles/IT24103381/testfile.txt
[2026-10-06 09:19:53] 127.0.0.1:44950 CMD GET testfile.txt
[2026-10-06 09:19:53] 127.0.0.1:44950 GET testfile.txt 16 bytes sent
[2026-10-06 09:20:09] 127.0.0.1:44950 CMD QUIT
[2026-10-06 09:20:09] 127.0.0.1:44950 disconnected (QUIT)
[2026-10-06 09:33:13] 127.0.0.1:52814 connected
[2026-10-06 09:33:27] 127.0.0.1:52814 CMD AUTH <token>
[2026-10-06 09:33:27] 127.0.0.1:52814 authentication OK
[2026-10-06 09:33:38] 127.0.0.1:52814 CMD MONITOR START 5000
[2026-10-06 09:33:38] 127.0.0.1:52814 MONITOR stream started -> udp port 5000 (every 2000 ms)
[2026-10-06 09:35:54] 127.0.0.1:40524 connected
[2026-10-06 09:36:10] 127.0.0.1:40524 CMD AUTH <token>
[2026-10-06 09:36:10] 127.0.0.1:40524 authentication OK
[2026-10-06 09:36:43] 127.0.0.1:40524 CMD AUTH <token>
[2026-10-06 09:36:43] 127.0.0.1:40524 authentication OK
[2026-10-06 09:36:58] 127.0.0.1:40524 CMD MONITOR START 5000

```

Figure 6.7: Five simultaneous Controllers

7. Testing Summary

The table lists the test cases for the mandatory features in section 2.2 of the brief. The repository also contains a test script, `test_381.sh` (written with AI assistance, see Section 8.3), that starts an Agent and runs most of these checks automatically with the real Controller. Cells shaded yellow must be filled in with the result I observed when I ran the test on my own machine (Pass or Fail).

#	Test case (what I did)	Expected result	Actual result
1	Command before AUTH (e.g. <code>SYSINFO</code>)	ERR 008 NOT_AUTHENTICATED SID:1833	Pass — got ERR 008 NOT_AUTHENTICATED SID:1833
2	AUTH wrong	ERR 001 AUTH_FAILED SID:1833	Pass — got ERR 001 AUTH_FAILED SID:1833, connection stayed open
3	AUTH OPS-3381	OK AUTHENTICATED SID:1833	Pass (Figure 3.1, log in Figure 4.2)
4	<code>SYSINFO</code> after AUTH	OK SYSINFO <cpu> <mem_mb> <uptime> SID:1833	Pass (log in Figure 4.2)
5	<code>LISTPROC</code> after AUTH	OK PROCS pid:name,... SID:1833	Pass (Figure 3.1)
6	<code>EXEC DATE</code> and <code>EXEC HOSTNAME</code>	OK EXEC_RESULT <output> SID:1833	Pass (Figure 6.3)
7	<code>EXEC UPTIME</code> , <code>DISKFREE</code> , <code>WHOAMI</code>	OK EXEC_RESULT ...	Pass — all three returned OK EXEC_RESULT with SID:1833
8	<code>EXEC rm</code> and <code>EXEC DATE;id</code>	ERR 002 COMMAND_NOT_ALLOWED SID:1833	Pass — both returned ERR 002 COMMAND_NOT_ALLOWED SID:1833
9	Unknown command <code>FOO</code>	ERR 009 UNKNOWN_COMMAND SID:1833	Pass — got ERR 007 UNKNOWN_COMMAND SID:1833
10	<code>PUT testfile.txt</code> (16 bytes)	OK FILE_RECEIVED testfile.txt; file in agentfiles/IT24103381/	Pass (Figures 6.4, 4.3)
11	<code>GET testfile.txt</code> after the <code>PUT</code>	OK FILE_SEND testfile.txt 16 + 16 bytes; dl_testfile.txt identical (cmp)	Pass (Figure 6.4) [Pass — cmp reported no differences; dl_testfile.txt identical]
12	<code>PUT/GET</code> of a 3 MB binary file	Downloaded file byte-for-byte identical (cmp)	Pass — downloaded file byte-for-byte identical (cmp returned no differences)
13	<code>GET nowhere.txt</code>	ERR 005 FILE_NOT_FOUND SID:1833	Pass — got ERR 005 FILE_NOT_FOUND SID:1833
14	<code>PUT</code> of an 11 MB file	ERR 004 FILE_TOO_LARGE; next command on the same connection still works	Pass — got ERR 004 FILE_TOO_LARGE SID:1833; next command on same connection worked
15	<code>PUT ../evil 5</code> followed by 5 bytes	ERR 003 BAD_FILENAME; the 5 bytes are not read as a command	Pass — got ERR 003 BAD_FILENAME SID:1833; the 5 trailing bytes were not read as a command

#	Test case (what I did)	Expected result	Actual result
16	Partial line and several lines in one packet	Commands are reassembled / split correctly; one reply per command	Pass — commands reassembled correctly; one reply per command
17	MONITOR START 5000	OK MONITOR_STARTED; UDP datagrams with SID:1833 about every 2 s	[Pass — OK MONITOR_STARTED SID:1833; UDP datagrams every ~2 s with SID:1833] (Figure 6.5)
18	MONITOR STOP	OK MONITOR_STOPPED; datagrams stop	[Pass — OK MONITOR_STOPPED SID:1833; datagrams stopped] (Figure 6.5)
19	QUIT	OK BYE SID:1833; log shows “disconnected (QUIT)”	Pass (Figures 3.1, 4.2)
20	Six simultaneous Controllers	All six authenticated and served correctly	[Pass — all six authenticated and served correctly; log shows 6 CONNECTED lines (Figure 6.7)]
21	Controller killed without QUIT	Agent keeps running; the next Controller is served	[Pass — Agent kept running; the next Controller was served normally]
22	Log content	Timestamps, commands, transfers, disconnects; token never written	Pass (Figure 4.2)

The automated script `test_381.sh` covers most of the tests above (1–2, 7–9, 12–18 and 20–21)

8. Design Rationale and Assumptions

8.1 Key design decisions

- **Thread per connection** (Section 2.3): simplest correct model for the brief's requirement of at least five simultaneous Controllers.
- **One framing layer for everything**: a single per-session buffer is used both for lines and for file bytes, so there is exactly one place that decides where a line or a file ends.
- **Reply helper adds the SID tag**: `reply()` is the only function that writes responses, so every OK and ERR line ends with `SID:1833`.
- **Fixed EXEC table**: the Controller chooses a name, never a command, so no shell injection is possible and the whitelist cannot be extended by accident.
- **Safe uploads**: restricted file names, a 10 MB limit, a temporary file renamed on completion, and draining of rejected uploads to keep the stream aligned.
- **UDP monitor as a separate thread per session**: independent of the TCP command handling, stopped quickly through an atomic flag and joined on exit.
- **Real values from /proc** with a simulated fallback, as the brief allows; the process list is read directly from /proc instead of running `ps`.
- **Consistent error codes**: 001 AUTH_FAILED, 002 COMMAND_NOT_ALLOWED, 003 BAD_FILENAME, 004 FILE_TOO_LARGE, 005 FILE_NOT_FOUND, 006 BAD_REQUEST, 007 MONITOR_ALREADY_RUNNING, 008 NOT_AUTHENTICATED, 009 UNKNOWN_COMMAND, 010 LINE_TOO_LONG, 011 STORAGE_ERROR, 012 INTERNAL_ERROR.
- **Token kept out of the log**: the log line for AUTH shows `<token>` instead of the value.

8.2 Assumptions where the brief leaves a choice

- The registration number IT24103381 follows the example format, so the first four digits of the numeric part are 2410 and the last four are 3381.
- The UDP monitoring interval is **2 seconds**, as the brief asks me to state it.
- Error codes (above) are my own choice; the brief only requires the format `ERR <code> <REASON_CODE>` with self-explanatory reasons.
- The maximum upload size is 10 MB, and file names are limited to letters, digits, `.`, `_` and `-`.
- PUT from the Controller uses the base name of the local path; GET saves to `d1_<name>` unless a second argument gives a name.
- Uploading a file with an existing name replaces it (the last completed upload wins).
- The monitoring datagrams are sent to the IP address of the Controller's TCP connection, on the UDP port given in `MONITOR START`; one stream per session.

- The process list uses `pid:name` pairs separated by commas, which the brief allows (“process names/PIDs”).

8.3 Use of AI assistance

I used an AI assistant (Claude, by Anthropic) while working on Part 1. It generated the initial implementation of the Agent, Controller and Makefile from the assignment brief and my registration number, and helped me with my Linux environment and with preparing this report. I compiled, ran and checked the program myself on my own machine (CentOS Stream 10, gcc 14.4.1), took all execution screenshots from my own runs, and recorded the AI interactions in my prompt log and my reflection.

9. Optional Extension and Limitations

Extension implemented: transfer throughput. The Controller reports the throughput in bytes per second for every PUT and GET. It measures the time with `clock_gettime(CLOCK_MONOTONIC)`: for a PUT from just before the header is sent until the Agent's OK FILE_RECEIVED arrives, and for a GET from sending the command until the whole file has been saved. Figure 6.4 shows the output, for example [PUT] 16 bytes in 0.0418 s = 383 bytes/s.

Limitations.

- The control channel is plain TCP, so the token and the data are not encrypted (TLS was not implemented).
- Thread per connection does not scale to thousands of clients; a thread pool or `epoll` would be needed for that.
- There is one shared authentication token for all Controllers and no per-user permissions.
- UDP monitoring datagrams are unauthenticated and may be lost; this is acceptable because they only carry statistics.
- There is no persistent history of SYSINFO/LISTPROC and files are limited to 10 MB.

10. Build and Run Instructions

Requirements: Linux with gcc and make (tested on CentOS Stream 10 with gcc 14.4.1).

Build (from the folder containing the three source files):

```
make -f Makefile_381
```

Run the Agent (terminal 1; it listens on TCP port 9410 and creates `./agentfiles/IT24103381/` and `remoteops_IT24103381.log`):

```
./agent_381
```

Run a Controller (terminal 2; default host 127.0.0.1, port 9410):

```
./controller_381 [host] [port]
```

Then type, for example: `AUTH OPS-3381, SYSINFO, LISTPROC, EXEC DATE, PUT testfile.txt, GET testfile.txt, MONITOR START 5000, MONITOR STOP, QUIT.`

Run the automated checks (optional): `./test_381.sh`.